

여러 AI를 한 화면에서, tmux로 서로 평가·검증하기

Claude·GPT·North를 한 터미널에서 동시에 굴려 교차검증하는 워크플로

여러 CLI 코딩 에이전트를 `tmux` 한 화면에 띄워 서로의 결과를 평가·검증하는 방법을 혼자 익힌다. Claude **Agent Teams**로 리뷰어를 팀메이트로 띄우고, 합의를 코드로 집계해 단독 풀이의 사각지대를 메운다.

대상 터미널·AI 코딩 도구를 막 시작한 입문자

구성 Claude(팀 리드) + opencode 리뷰어 GPT·North(Agent Teams 팀메이트)

출처 tmux 공식 문서·Claude Code 문서·Anthropic Engineering·cross-verify 템플릿

이 자료의 큰 흐름 — 무엇·왜·어떻게·왜 나온가

이 묶음은 네 갈래 질문을 차례로 풀어간다. 끝까지 따라오면 도구·구조·흐름·근거를 모두 손에 쥘다.

01

이걸 뭐라 부르나

여러 에이전트를 한 화면에서 함께 굴려 **서로 평가·검증**하는 것을 **멀티에이전트 오케스트레이션**이라 부른다. 먼저 이름과 그림부터 잡는다.

02

왜 하필 tmux인가

터미널 창을 여러 개 띄우는 것과 무엇이 다른지 본다. 답은 **세션 영속성**과 다른 화면을 **코드로 조종**하는 제어다.

03

어떻게 돌아가나

리드가 **Agent Teams**로 리뷰어를 팀메이트로 띄워 `SendMessage`로 비평을 받고, `consensus.py`로 합의를 집계한 뒤 두 축으로 채점하는 흐름을 익힌다.

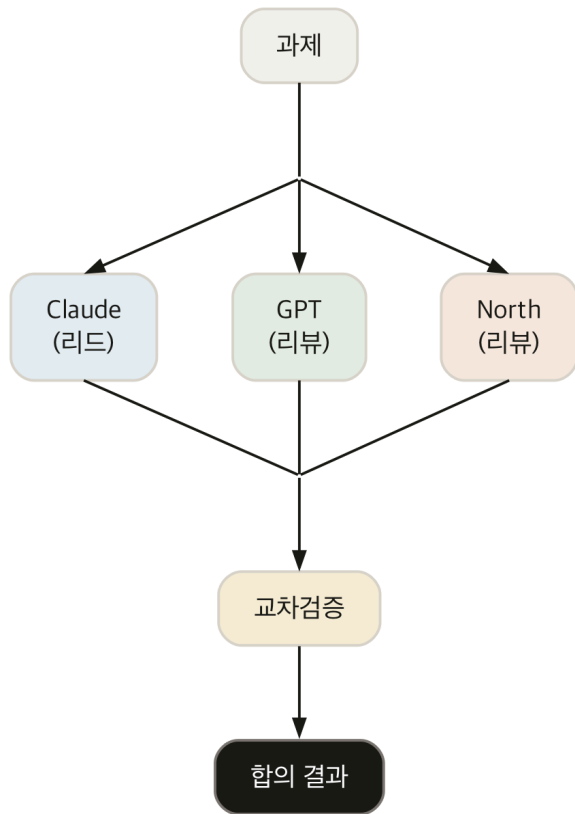
04

왜 단독보다 나온가

계보가 다른 모델로 **독립 검증**하면 한 모델의 사각지대를 메운다. 결과는 **명세 기준선**과 **교차검증이 추가로 잡은 양**, 두 축으로 본다. 뒤에서 **언제 쓰고 피할지(비용)**도 다룬다.

여러 CLI 코딩 에이전트를 동시에 굴려 서로 평가·검증

CLI 코딩 에이전트 — 터미널에서 명령으로 도는 AI 코딩 도구(예: `claude` 명령, `opencode`). 이런 에이전트 여럿을 한 자리에 모은다.



01 무엇을 하나

Claude·GPT·North 같은 서로 다른 CLI 코딩 에이전트를 `tmux` 한 화면에 나란히 띄운다. **Agent Teams**로 리뷰어를 팀메이트로 띄우면 각자 페인에 자동 배치된다.

02 어떻게 도나

같은 과제를 각자 **독립으로** 풀고 그 결과를 **서로 평가·검증**한다. 한 모델의 답을 다른 계보의 모델이 다시 따져보는 구조다.

03 왜 중요한가

혼자 푸는 모델은 자기 약점을 스스로 보기 어렵다. 여럿이 교차로 검증하면 **합의는 확신**이 되고, **엇갈리는 답은 다시 살필 신호**가 된다.

04 어디에 쓰나

미묘한 정확성이나 보안이 걸린 코드 검토에 맞다. 이어지는 슬라이드에서 도구(`tmux`)·구조·검증 루프를 차례로 본다.

터미널 에뮬레이터 vs 터미널 멀티플렉서



한 터미널 창을 tmux가 좌우 두 pane으로 나눈 실제 화면. 출처: Wikimedia Commons, Demo-tmux.png, CC BY-SA 4.0

01 그림에서 보이는 것

터미널 한 창 안에서 tmux가 화면을 좌우 두 **pane**으로 쪼개 각각 다른 파일을 띄운 실제 화면이다. 맨 아래 초록 띠가 tmux **status line**이다 — 창 하나가 곧 여러 작업 공간이 된다.

02 터미널 에뮬레이터

iTerm2·Terminal.app처럼 검은 터미널 창 자체를 그려주는 프로그램이다. 창 하나가 곧 셸 하나이고, 창을 여러 개 띄워도 서로를 모른다.

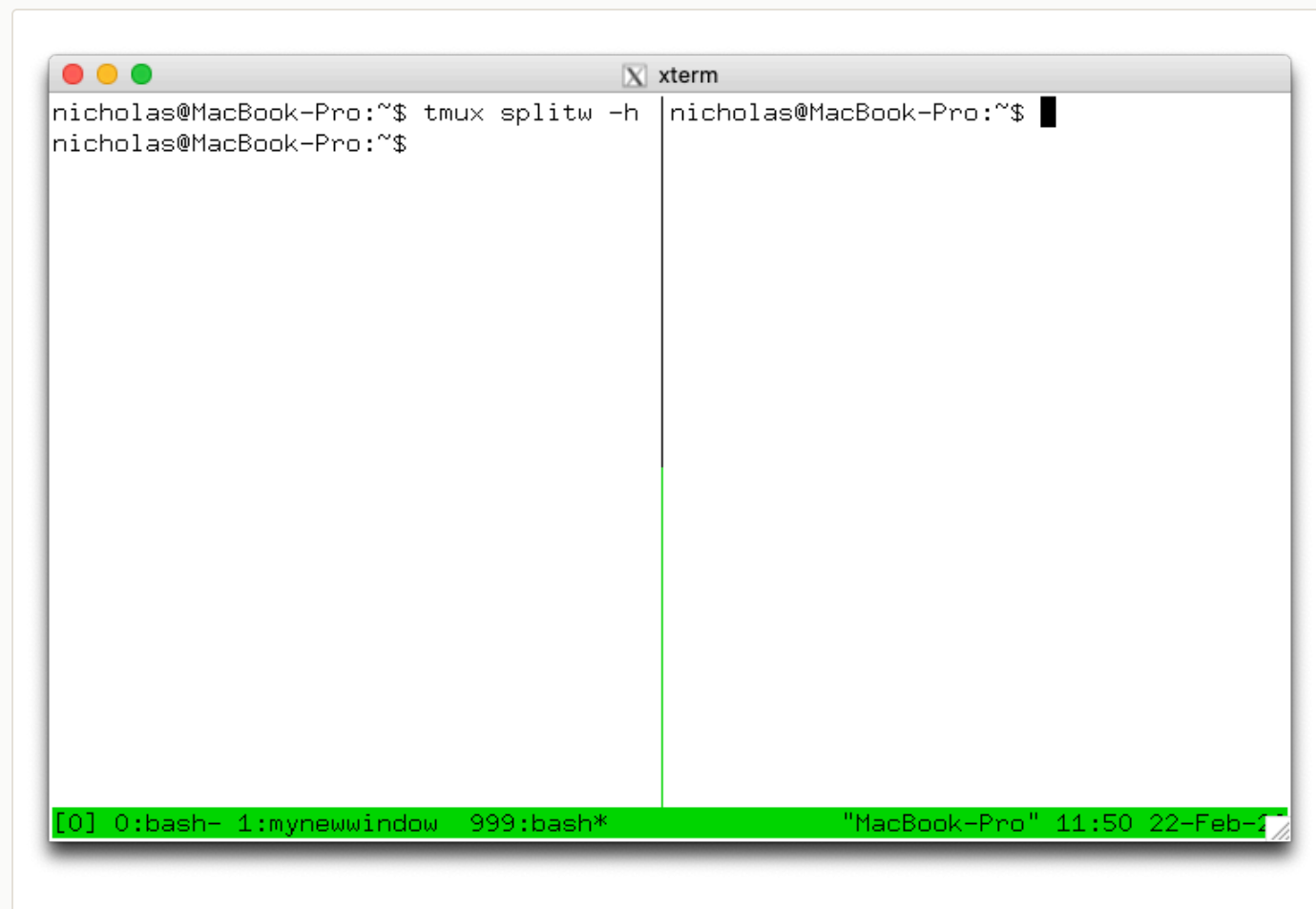
03 터미널 멀티플렉서(tmux)

에뮬레이터 **안에서** 돌며 그 한 창을 여러 **pane**으로 쪼개고 세션을 켜다. 오픈소스(BSD). 다른 pane을 코드로 조종하고 세션을 백그라운드에 살려둔다.

04 왜 이 구분이 중요한가

tmux는 에뮬레이터를 대체하지 않고 **그 안에 얹힌다**. 어떤 에뮬레이터 위에서도 화면 분할·세션 영속·제어를 더해 멀티에이전트의 토대가 된다.

터미널 창 여러 개로는 왜 안 되나 — 영속성과 제어



tmux splitw -h 명령으로 한 창을 좌우 분할한 실제 화면 — 명령 기반 제어. 출처: tmux 공식 위키(github.com/tmux/tmux/wiki), BSD

01 그림에서 보이는 것

`tmux splitw -h` 명령 한 줄로 창이 좌우 두 pane으로 갈라진 실제 화면이다. 마우스가 아니라 **명령으로** 화면을 제어한다는 점이 핵심이다.

02 왜 창 여러 개는 안 되나

독립 터미널 창은 OS가 붙여준 **전용 입출력 통로(PTY)**가 따로라, 옆 창에 키를 넣거나 화면을 읽을 통로가 없다.

03 tmux는 어떻게 되나

한 서버가 통로들을 모아 줘서

`split-window` · `send-keys` 로 **명령을 주입**하고

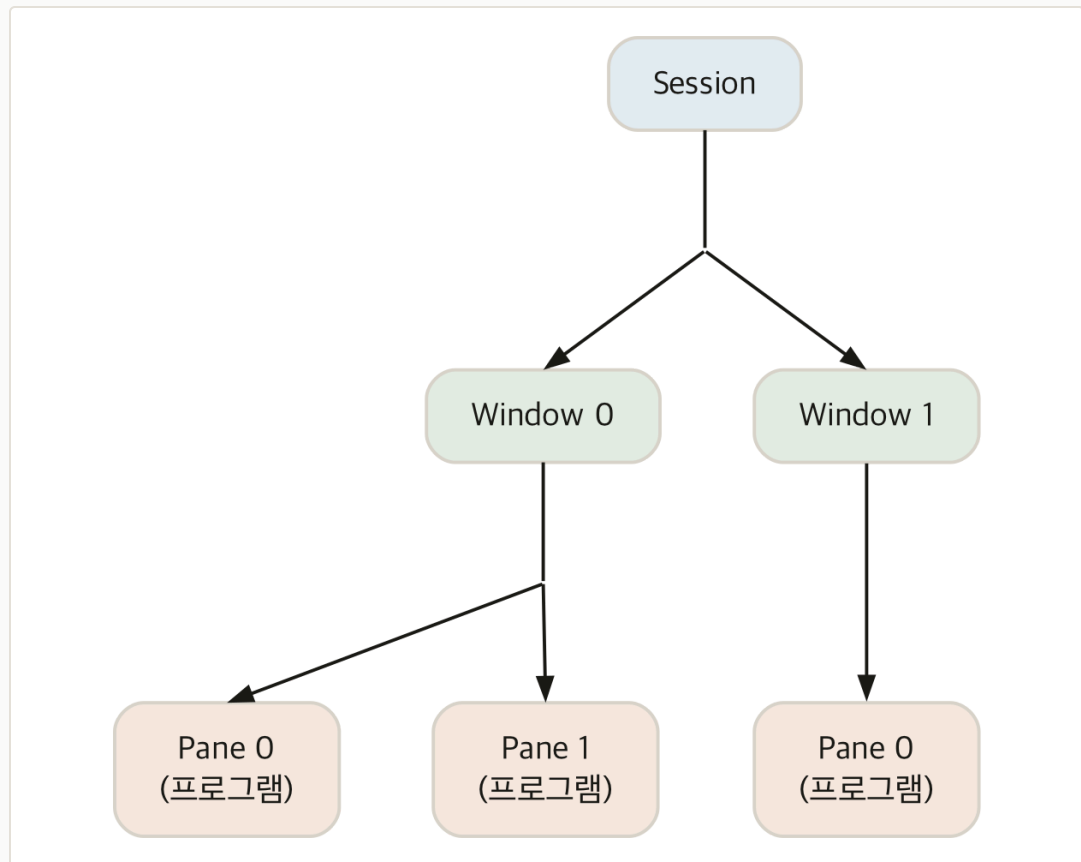
`capture-pane` 으로 화면을 **회수**한다. 세션은 detach해도 영속한다.

04 어디에 쓰나

이 화면 제어를 직접 쓰지 않고 Claude **Agent Teams**가 자동화한다. 그 토대가 tmux의 세션 영속성과 pane 제어다.

3계층 구조 — session > window > pane

pane — 실제 프로그램(셀·에이전트)이 도는 가장 작은 칸. 이 칸들이 모여 window, window들이 모여 session이 된다.



01 무엇을 보나

tmux는 세 겹의 포함 관계다. 그림에서 맨 위 **session** 아래로 **window**, 그 아래 **pane**이 가지처럼 이어진다.

02 어떻게 나뉜나

프로그램은 모두 **pane**에서 돈다. 각 pane은 하나의 **window**에 속하고, 여러 window가 묶여 하나의 **session**을 이룬다.

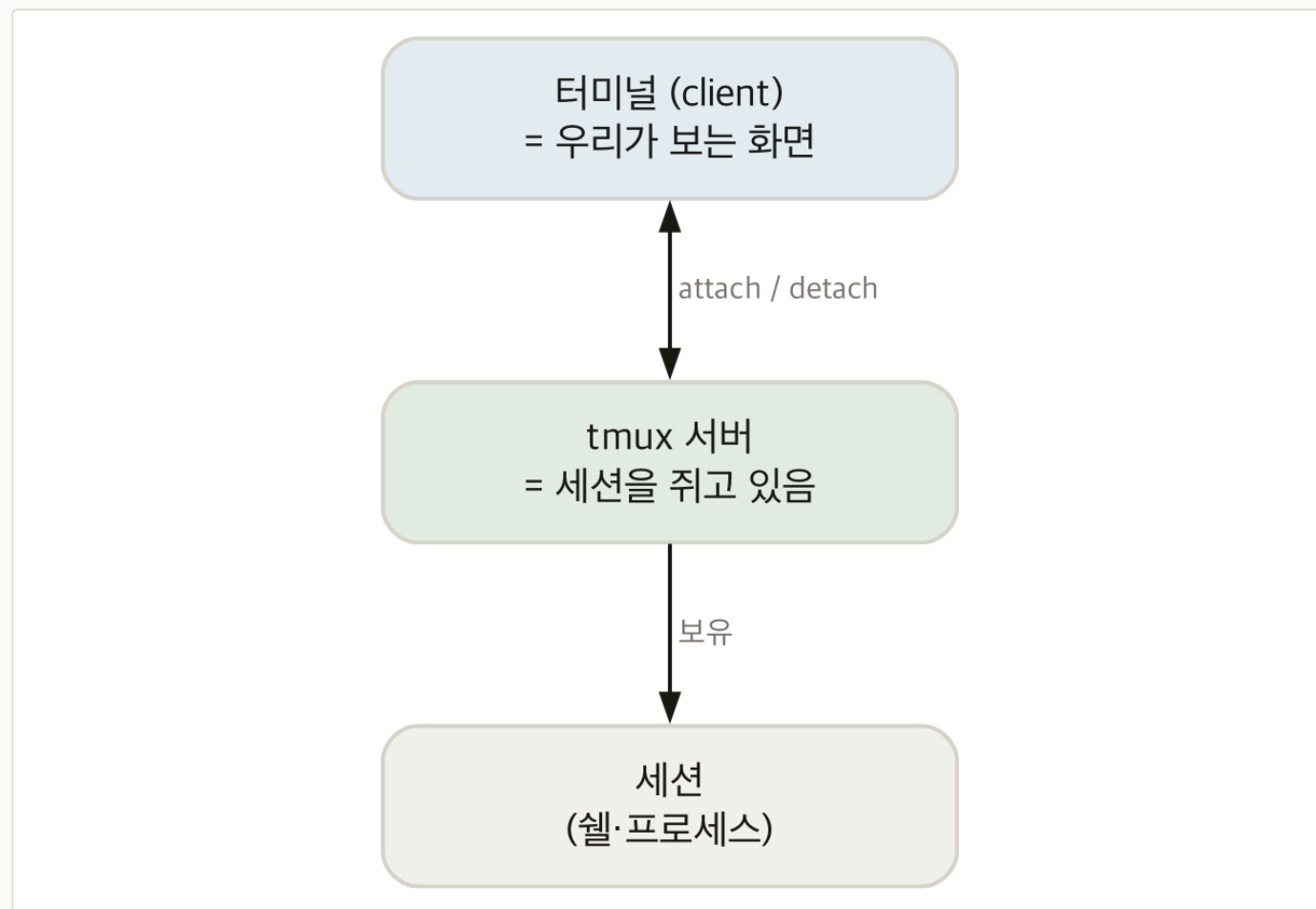
03 왜 중요한가

에이전트 하나를 **하나의 pane**에 두면, 같은 window 한 화면에서 여러 에이전트를 동시에 보고 조종한다. 주제별로 window를 나눠 정리할 수도 있다.

04 어디에 쓰나

Claude 리드는 **첫째** pane, GPT 리뷰어는 **둘째**, North 리뷰어는 **셋째** 칸으로 **역할별 자리**를 정해 한 session에 모아 굴린다.

화면과 세션이 분리된다 — detach 후 다시 attach



tmux 클라이언트(화면)와 백그라운드 서버(세션 보유)의 분리 — detach-attach 흐름.

01 무엇을 보나

tmux는 클라이언트와 서버가 나뉜다. 우리가 보는 터미널 화면이 클라이언트, 셸·프로세스를 실제로 들고 있는 쪽이 백그라운드 서버다.

02 어떻게 동작

화면을 닫거나 SSH가 끊겨도 세션은 서버가 그대로 쥐고 있다. 화면(클라이언트)만 떨어질 뿐(detach), 서버 안 작업은 계속 돈다.

03 왜 중요

다른 자리·다른 컴퓨터에서 `tmux attach` 로 그 서버에 다시 붙으면(attach) 끊긴 화면이 그대로 되살아난다. 일반 터미널은 닫으면 셸까지 같이 죽는다.

04 어디에 쓰나

오래 도는 에이전트·개발 서버를 끊임 없이 유지할 때.

tmux 실제 화면 — 한 창을 여러 페인으로



01 그림에서 보이는 것

한 터미널 창이 **세 개의 페인**으로 갈라져 있다. 위·아래·옆이 각각 독립된 셸로, 한 화면 안에 여러 작업이 동시에 들어선 모습이다.

02 어떻게 나뉜다

`%`로 좌우, `"`로 상하로 쪼갬 결과다. 페인 사이 **경계선**과 아래쪽 **상태표시줄**로 지금 어느 페인에 있는지 구분된다.

03 왜 중요한가

에이전트마다 페인 한 칸을 주면, 여러 에이전트의 출력을 **한눈에** 보면서 원하는 칸으로 옮겨 곧장 손댈 수 있다.

04 어디에 쓰나

리드·GPT·North를 세 칸에 늘어놓고 함께 굴릴 때 이 배치를 쓴다.

tmux 세션이 세 페인으로 분할된 실제 화면. 출처: Wikimedia Commons, "Tmux.png", Oxiq, 2011, BSD Unlimited License

설치와 첫 실행 — tmux · opencode · Agent Teams 켜기

macOS 기준 진입 절차다. tmux·opencode를 설치하고 쓸 모델의 provider를 인증한 뒤, Agent Teams를 켜고 tmux 안에서 claude를 띄우는 데까지 한다.

```
# 1) install tmux and opencode (macOS, Homebrew)
brew install tmux
brew install anomalyco/tap/opencode
# or: curl -fsSL https://opencode.ai/install | bash

# 2) authenticate a provider, then check what's available
# free North needs no key; for GPT, log in to OpenAI
opencode auth
opencode auth list    # which providers are authenticated
opencode models      # exact model identifiers for -m

# 3) turn on Agent Teams (.claude/settings.json)
# env: CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS = 1
# teammateMode: "tmux"

# 4) MUST start tmux first, then claude inside it
tmux
claude
```

01 brew install

tmux와 opencode를 Homebrew로 설치한다. opencode는 여러 모델을 한 CLI로 비대화 실행하는 도구로, `anomalyco/tap` 또는 공식 install 스크립트로 받는다.

02 provider 인증

provider는 모델 제공사다. 유료 모델(GPT 등)은 그 회사 계정 로그인에 있어야 호출되므로 `opencode auth` 로 OpenAI에 로그인하고, 무료 North는 키 없이 바로 쓴다. 실제 모델 식별자는 `opencode models` 로 확인해 `-m` 에 붙인다(이 자료의 식별자는 예시).

03 Agent Teams 켜기

`.claude/settings.json` 에 `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` 과 `teammateMode:"tmux"` 를 둔다. 이 템플릿엔 이미 켜져 있다. 이게 있어야 리드가 띄운 팀메이트가 각자 페인으로 보인다.

04 tmux 안에서 claude

팀메이트를 페인으로 보려면 반드시 **tmux 세션 안**(`$TMUX` 존재) 에서 `claude` 를 실행해야 한다. tmux 밖에서 띄우면 페인 자동 배치가 동작하지 않는다.

핵심 단축키 한눈에 — prefix는 Ctrl-b

xterm		[59/59]
Ctrl-b	Send the prefix key	
Ctrl-b Ctrl-o	Rotate through the panes	
Ctrl-b Ctrl-z	Suspend the current client	
Ctrl-b Space	Select next layout	
Ctrl-b !	Break pane to a new window	
Ctrl-b "	Split window vertically	
Ctrl-b #	List all paste buffers	
Ctrl-b \$	Rename current session	
Ctrl-b %	Split window horizontally	
Ctrl-b &	Kill current window	
Ctrl-b '	Prompt for window index to select	
Ctrl-b (	Switch to previous client	
Ctrl-b)	Switch to next client	
Ctrl-b ,	Rename current window	
Ctrl-b -	Delete the most recent paste buffer	
Ctrl-b .	Move the current window	
Ctrl-b /	Describe key binding	
Ctrl-b 0	Select window 0	
Ctrl-b 1	Select window 1	
Ctrl-b 2	Select window 2	
Ctrl-b 3	Select window 3	
Ctrl-b 4	Select window 4	
Ctrl-b 5	Select window 5	
[0] 0:[tmux]*		"MacBook-Pro" 11:13 22-Feb-1

tmux 키 바인딩 실제 목록(prefix Ctrl-b ?, 59개). 출처: tmux 공식 위키(github.com/tmux/tmux/wiki), BSD

01 그림에서 보이는 것

tmux에서 **prefix(Ctrl-b)** **?** 를 누르면 뜨는 실제 키 목록이다(총 59개). 각 줄은 **C-b** 다음에 누를 키와 그 동작이다 — 명령으로 띄운 진짜 도움말 화면이다.

02 어떻게 외우나

" 상하 분할 · **%** 좌우 분할 · **c** 새 window · **,** window 이름 변경 · 숫자로 그 window 선택. **prefix**를 먼저 누르고 키를 따로 누른다(동시누름 아님).

03 왜 중요한가

분할로 여러 에이전트를 **한 화면에** 늘어놓고, prefix+화살표로 pane을 오가며 즉시 손댄다. **d** 로 detach해 세션을 살려둔 채 자리를 비운다.

04 근거 구분

이 목록은 tmux가 직접 출력한 공식 키 바인딩이다. 버전·설정에 따라 일부 키가 다를 수 있어, 막히면 **prefix ?** 로 직접 확인한다.

세션 만들고 · 목록 보고 · 다시 붙기

세션을 다루는 가장 기본인 세 명령이다. 만들고(new), 목록을 확인하고(ls), 다시 붙는다(attach).

```
# create a named session
tmux new -s review

# list running sessions
tmux ls
# review: 3 windows (created ...)

# re-attach to a detached session
tmux attach -t review
```

01 tmux new -s NAME

`review` 라는 이름의 세션을 새로 만들고 그 화면에 붙는다. 이름을 주면 나중에 골라 붙기 쉽다.

02 tmux ls

현재 살아 있는 세션 목록을 보여준다. 떼어둔 세션도 여기 나오므로 살아 있는지 확인한다.

03 tmux attach -t NAME

떼어둔(detached) 세션에 다시 붙는다. 자리를 비웠다 돌아와 작업을 이어갈 때 쓴다.

Agent Teams — 팀메이트 스폰과 SendMessage 회신

Agent Teams — 리드가 띄운 팀메이트들이 각자 tmux pane에서 돌며 공유 task list로 일을 나누고 메시지로 소통하는 방식.

출처: Anthropic Claude Code 공식 문서(code.claude.com/docs, agent-teams)

01 그림에서 보이는 것

왼쪽 **Subagents**는 메인인 서브를 스폰→작업→결과→보고하는 일방 구조, 오른쪽 **Agent Teams**는 팀 리드가 **공유 task list**를 두고 팀메이트끼리 **서로 소통 (Communicate)**하는 구조다.

02 무엇을 하나

리드(Claude)가 **Agent** 도구로 **리뷰어**를 **팀메이트**로 스폰하면 각 팀메이트가 자기 pane에 자동 배치된다. 리드는 `opencode`를 직접 돌리지 않는다.

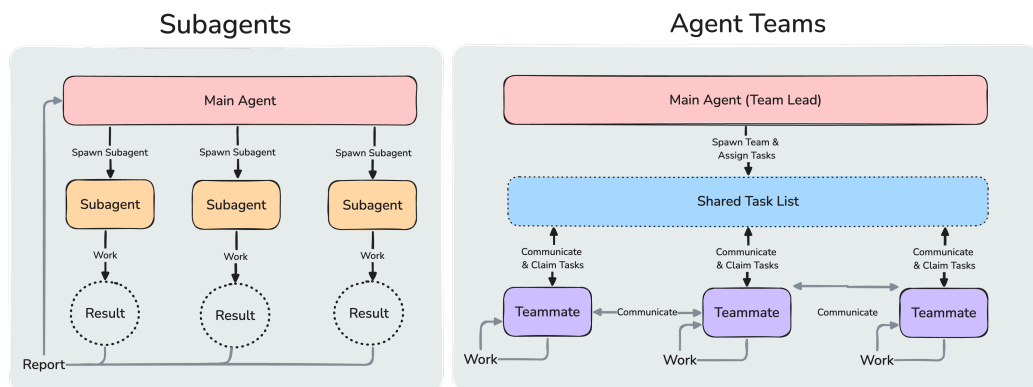
03 어떻게 회신하나

팀메이트의 일반 텍스트 출력은 리드에 안 보인다. 비평은 **오직**

`SendMessage(to: "team-lead")` 로만 전달된다(이 하네스에서 `to: "main"` 은 거부).

04 리드는 무엇을 하나

리드는 초안을 쓰고 팀메이트를 스폰한 뒤, 도착한 비평을 받아 **종합**한다. 실제 `opencode` 실행은 각 리뷰어가 자기 pane에서 한다.



Agent Teams로 리뷰어 스폰 + opencode read-only 비평

리드가 `Agent` 도구로 두 리뷰어를 한 메시지에서 동시에 스폰한다. 각 리뷰어는 자기 pane에서 opencode를 read-only로 한 번 돌려 비평한 뒤 `SendMessage`로 회신한다.

```
# 1) Agent Teams ON (.claude/settings.json)
# CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS = 1
# teammateMode: "tmux"

# 2) lead spawns 2 reviewers (Agent tool, one message)
# -> each gets its own pane automatically

# 3) each reviewer runs ONE foreground opencode in its pane:
# GPT reviewer:
timeout 120 opencode run -m openai/gpt-5.4 \
  "read SPEC.md and solo/solution.py, critique vs spec"
# North reviewer (code inlined, no tools):
timeout 60 opencode run -m opencode/north-mini-code-free \
  "do not read files or use tools; here is the code: ..."

# 4) opencode.json keeps reviewers read-only:
# edit: deny, bash: deny, webfetch: deny
# read: allow, grep: allow, glob: allow
```

01 Agent로 동시 스폰

리드는 `Agent` 도구 한 메시지로 GPT-North 리뷰어를 함께 띄운다. 각 팀메이트가 자기 pane에 자동 배치되고, 거기서 자기 opencode를 돌린다. 리드는 opencode를 직접 실행하지 않는다.

02 SendMessage로 회신

팀메이트의 일반 출력은 리드에 안 보인다. 비평은

`SendMessage(to:"team-lead")` 로만 보낸다(이 하네스에서 `to:"main"` 은 거부됨). 한쪽만 와도 받은 것으로 진행하고 늦게 오면 그때 반영한다.

03 코드 인라인 + 도구 금지

North 프롬프트엔 "파일 읽지 말고 도구 쓰지 마라"를 넣고 코드를 인라인으로 붙인다. 안 넣으면 opencode build 에이전트가 레포를 grep-재시도하며 헛돌아 늘어진다. 인라인+no-tools면 한 패스 약 10초다(실측).

04 read-only 안전

`opencode.json` 이 `edit: bash: webfetch: deny, read: grep: glob: allow`로 둔다. 리뷰어는 분석 전용이라 파일을 고치지 못한다. 모델 식별자 (`openai/gpt-5.4` 등)는 이 자료 기준 예시다.

cross-verify 한 흐름 — 기준선 → 초안 → 비평 → 합의 → 종합 → 두 축 채점

앞의 조각을 한 워크플로로 잇는다. 명세 기준선 테스트를 먼저 고정하고, 초안을 쓰고, 리뷰어 비평을 합의로 집계해 종합한 뒤, 두 축으로 채점한다.

```
# test_spec.py -- importlib.import_module(os.environ['SPEC_MODULE'])

# 2) write the draft
# solo/solution.py

# 3) spawn GPT + North reviewers (Agent Teams)
# collect critiques via SendMessage(to="team-lead")

# 4) normalize to findings.json, then aggregate consensus
python3 consensus.py findings.json
# 2+ models on the SAME repro_input -> consensus

# 5) accepted cases go to a SEPARATE file (not the baseline)
# test_crossverify.py

# 6) write the synthesis
# crossverify/solution.py

# 7) score BOTH modules on BOTH test files
for mod in solo.solution crossverify.solution; do
  SPEC_MODULE=$mod pytest test_spec.py test_crossverify.py -q
```

01 기준선 먼저 고정

초안·비평을 보기 전에 `SPEC.md` 만 근거로 `test_spec.py` 를 짜 둔다.
수정본 뒤에 짜면 점수가 자기충족이 되므로, 이 파일은 비평을 본 뒤에도 **절대** 수정하지 않는다.

02 엿지는 별도 파일로 분리

리뷰어가 잡은 케이스는 기준선이 아니라 `test_crossverify.py` 에 넣는다.
그래야 "solo도 명세 기준은 통과(=무능하지 않음)"와 "교차검증이 **추가로** 잡은 것"이 분리돼 보인다.

03 consensus.py 결정론

비평을 `findings.json` 으로 정규화해 `consensus.py` 로 집계한다. **반례**
입력 문자열 완전일치로 2+ 모델이 겹치면 합의(우선 반영), 1 모델이면 단독
(참고)이다. LLM 판단은 끼지 않는다.

04 상보적 커버리지·종합

두 모델이 **서로 다른** 버그를 잡으면 합의 0건이 정상이고, 이는 **상보적**
커버리지다. 리드가 합의·단독·자기 판단을 종합해
`crossverify/solution.py` 를 만들고, 두 모듈을 두 테스트로 채점한다.

실습 템플릿 폴더 구조 — 어떤 파일이 무슨 역할인가

cross-verify-template — `SPEC.md` 과제 하나를 Claude·GPT·North가 교차검증하도록 미리 짜인 실습 폴더. 윗부분은 내가 직접 두는 파일, 아랫부분은 실행하면 자동으로 생긴다.

cross-verify-template/ · 직접 두는 파일	
SPEC.md	★ 과제 정의 — 여기만 바꾼다
consensus.py	합의 집계(결정론)
opencode.json	리뷰어 read-only 권한
.claude/settings.json	Agent Teams 켜기
.claude/skills/.../SKILL.md	교차검증 절차
.claude/agents/*-reviewer.md	GPT·North 리뷰어 정의
↓ 실행하면 자동 생성	
solo/solution.py	단독 초안
crossverify/solution.py	교차검증 종합본
test_spec.py	명세 기준선(고정·불변)
test_crossverify.py	교차검증이 추가로 잡은 엡지
findings.json	비평 정규화 입력

01 무엇을 두나

`SPEC.md` 만 내 과제로 바꾸면 끝이다. `consensus.py` (합의 집계)·`opencode.json` (리뷰어 read-only)·`.claude/` (Agent Teams·절차·리뷰어 정의)는 그대로 동작한다.

02 누가 만지나

리드 **Claude**가 초안·종합·테스트를 쓰고, **GPT·North** 리뷰어는 read-only로 비평만 보낸다. 그래서 `opencode.json` 이 edit·bash를 막아 둔다.

03 무엇이 자동 생성되나

실행하면 `solo/` (단독 초안)·`crossverify/` (종합본)·`test_spec.py` (명세 기준선)·`test_crossverify.py` (교차검증 엡지)·`findings.json` (비평 정규화)이 생긴다.

04 왜 이렇게 나누나

직접 두는 입력과 자동 산출물을 분리하면, 어느 단계에서 무엇이 만들어지는지 폴더만 봐도 흐름이 읽힌다. 다음 장의 흐름·역할 도식이 이 파일들을 단계에 연결한다.

신호는 tmux, 데이터는 파일

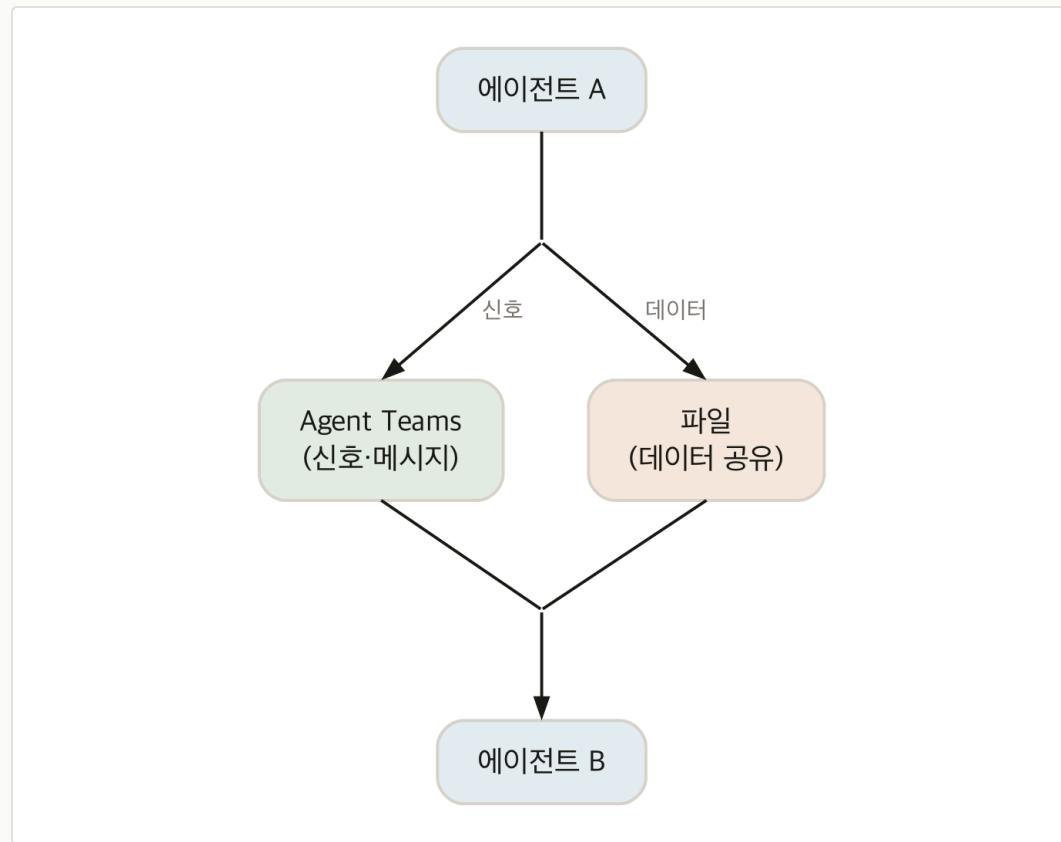
에이전트끼리 무엇을 메시지로 주고받고 무엇을
파일로 주고받을지 가르는 한 줄 원칙이 있다.

신호와 명령은 Agent Teams 메시지로 보낸다. `SendMessage` 로 "비평 보냄", "검토 끝" 같은
짧은 제어를 팀메이트끼리 주고받는다. 가볍고 즉각적인 트리거에 맞다.

결과와 데이터는 파일로 주고받는다.

`SPEC.md` · `solo/solution.py` · `test_*.py` · `findings.json` 처럼 양이 많고 **다시
읽어야 할 산출물**은 공유 디스크에 파일로 남기고, 메시지로 "그 파일 보라"는 신호만 보낸다.

이렇게 **경로를 나누면** 화면 출력이 깨지거나 잘려도 데이터가 안전하고, 누가 무엇을 만들었는지
파일로 남아 추적된다. 그림에서 신호는 메시지 채널, 데이터는 파일 경로로 표시했다.

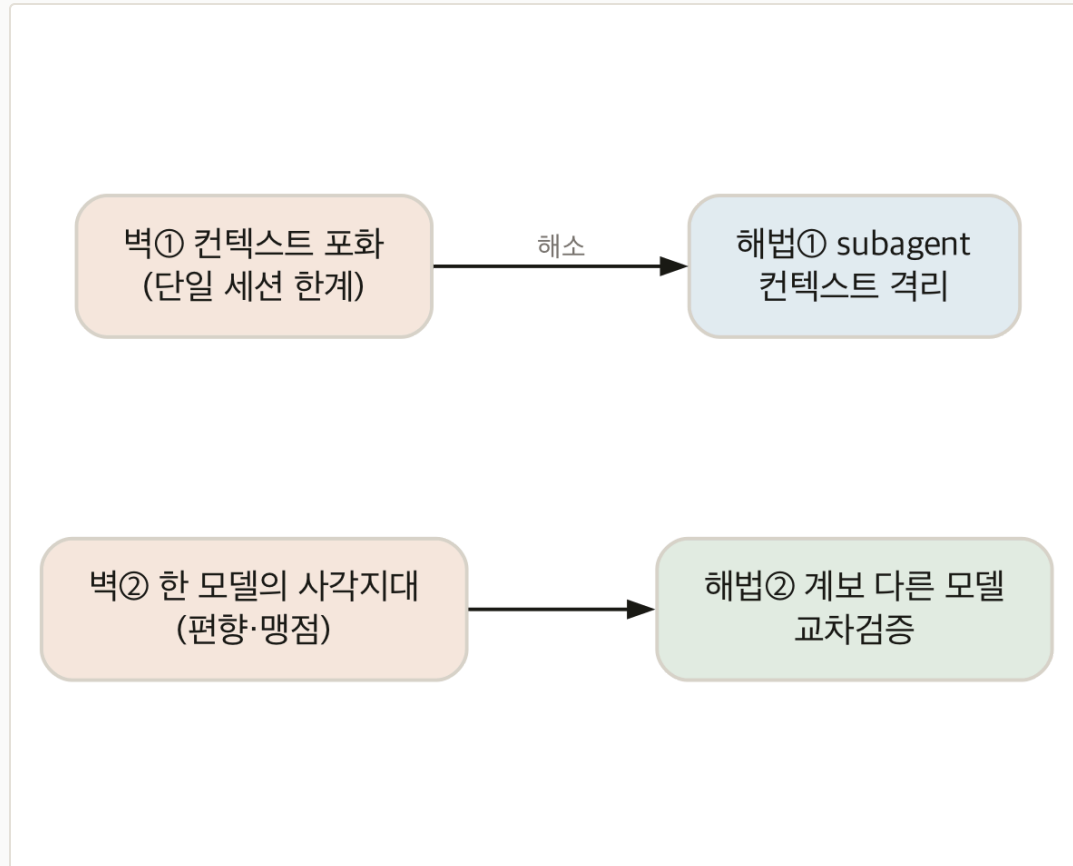


제어 신호는 Agent Teams 메시지로, 데이터(결과물)는 파일로 흐른다.

"신호는 tmux, 데이터는 파일." — 설계 원칙

단일 에이전트의 한계 — 컨텍스트 포화와 한 모델의 사각지대

컨텍스트 포화 — 다시 볼 일 없는 내용이 컨텍스트 창을 채워 정작 중요한 맥락을 밀어내는 상태. **사각지대** — 한 모델이 늘 같은 자리에서 놓치는 맹점.



01 무엇을 보나

그림은 단일 에이전트가 부딪히는 **두 벽**과 각 벽의 해법을 1:1로 잇는다. 왼쪽 두 벽(포화·사각지대)이 오른쪽 두 해법(격리·교차검증)과 화살표로 짝지어진다.

02 벽① 컨텍스트 포화 → 해법① 격리

한 대화에 탐색 로그·긴 파일이 쌓이면 중요한 맥락이 **밀려난다**. 이 벽은 Claude Code의 **subagent 컨텍스트 격리**로 푼다. 겹가지 작업을 별도 컨텍스트에서 처리해 메인을 보존한다.

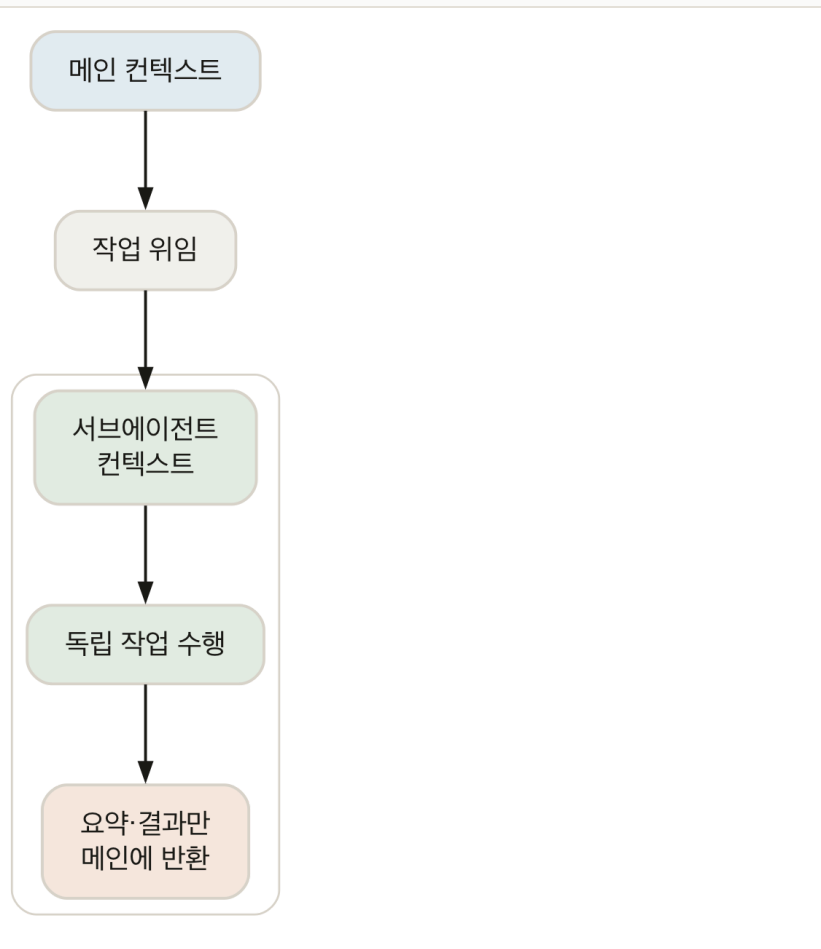
03 벽② 한 모델의 사각지대 → 해법② 교차검증

같은 계보 모델은 비슷한 데이터로 학습해 **비슷한 실수**를 반복한다. 이 벽은 **계보가 다른 모델로 교차검증**해 메운다. 한 모델이 놓친 자리를 다른 모델이 짚는다.

04 왜 1:1인가

두 벽은 성격이 달라 해법도 다르다. 격리는 **한 모델 안**의 포화를, 교차검증은 **모델 사이**의 맹점을 다룬다. 다음 두 슬라이드가 각각의 해법이다.

컨텍스트 격리 — 각자 자기 컨텍스트 창에서 독립 작동



Claude Code subagent의 컨텍스트 격리 구조.

01 그림에서 보이는 것

메인 대화와 **별도의 컨텍스트 창**에서 subagent가 따로 일한다.
옆길 작업이 메인 대화로 흘러들지 않는다.

02 어떻게 동작하나

각 subagent는 자기만의 컨텍스트 창에서 **커스텀 프롬프트·도구·권한**을 갖고 독립으로 작동한다. 끝나면 **요약만** 메인에 돌려준다.
Agent Teams의 팀메이트도 마찬가지로 자기 컨텍스트에서 돈다.

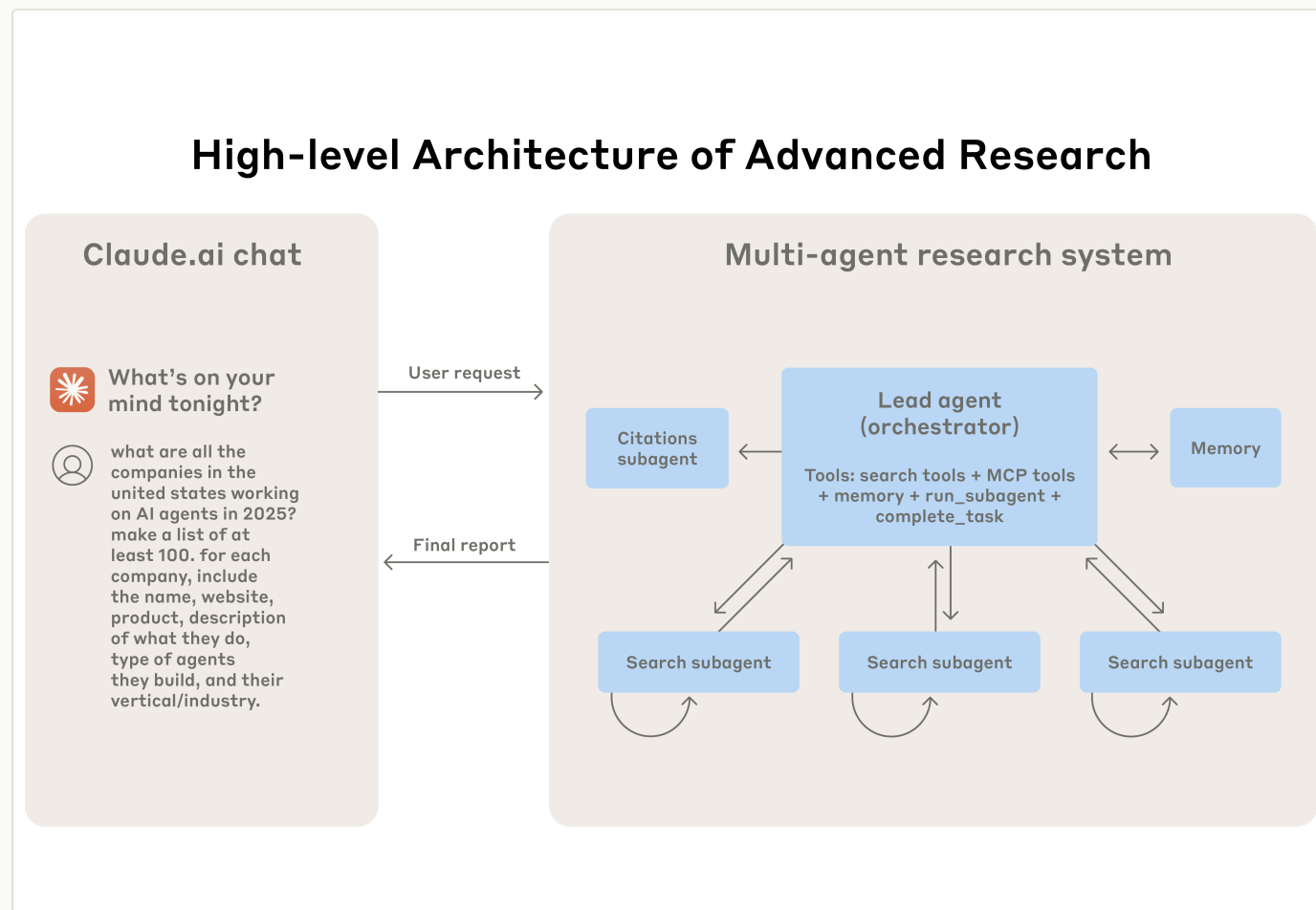
03 왜 중요한가

탐색·구현으로 메인 대화가 넘치는 것을 막아 **메인 컨텍스트를 보존**한다. 이는 컨텍스트 포화를 막는 해법이다.

04 어디에 쓰나

긴 검색·코드 탐색을 걸가지로 돌릴 때 쓴다.

계보가 다른 모델로 독립 검증 — 모델 다양성



오케스트레이터-위커 아키텍처. 출처: Anthropic Engineering, "How we built our multi-agent research system"

01 그림에서 보이는 것

사용자 요청이 리드(오케스트레이터)로 들어가고, 리드가 여러 독립 서브에이전트에 분담시킨 뒤 결과를 모아 최종 보고서를 낸다.

02 어떻게 동작하나

서브에이전트들은 각자 독립으로 같은 문제를 다룬다. 우리 구성은 여기에 **계보가 다른 모델**(Claude·GPT·North)을 끼워 다양성을 더한다.

03 왜 중요한가

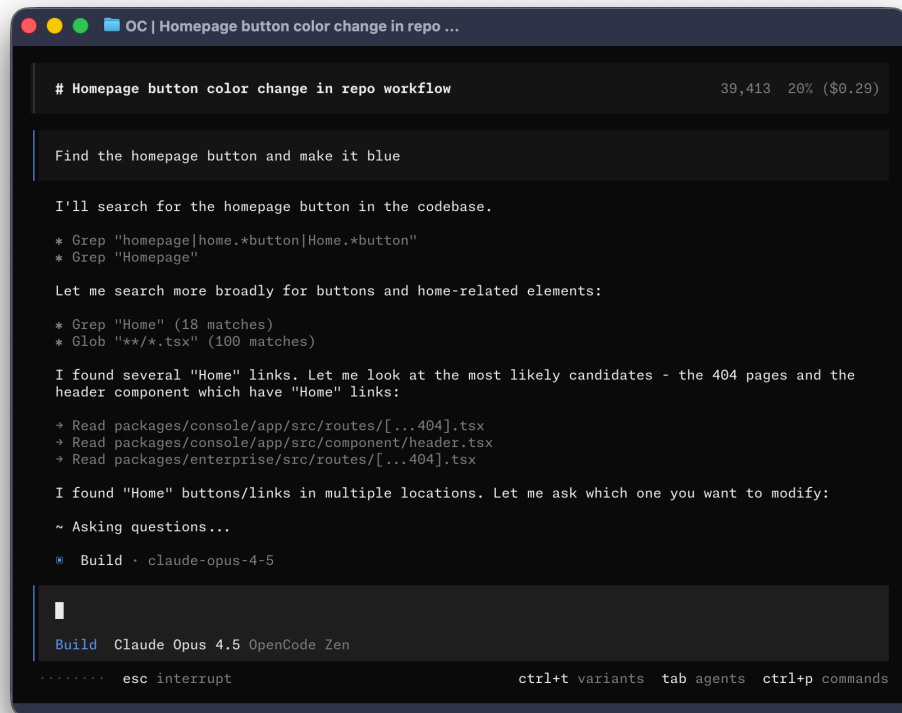
같은 계보 모델은 같은 함정에 빠지기 쉽다. **계보를 섞어** 교차검증하면 한 모델의 사각지대를 메워 **놓치는 오류가 준다**.

04 어디에 쓰나

정확성이 까다로운 검토에 쓴다.

Claude(리드) + opencode로 돌리는 GPT·North 리뷰어

opencode — provider/model 형식으로 여러 모델을 비대화로 실행하는 CLI. 리뷰어가 이걸 거쳐 GPT·North를 호출한다. 사진은 opencode를 터미널에서 실행한 실제 화면이다.
출처: opencode 공식 저장소(github.com/sst/opencode), MIT



01 그림에서 보이는 것

opencode를 터미널에서 띄운 실제 TUI다. 위에 작업 프롬프트, 아래로 모델이 grep·glob로 코드를 훑고 답하는 과정, 맨 아래 줄에 **실행 모델명**(예: Claude Opus)이 뜬다.

02 우리 구성

맨 위 **Claude**가 리드, **opencode 게이트**를 거쳐 **GPT·North** 리뷰어로 갈라진다. 리드는 초안·종합·채점을 쓰고, 두 리뷰어는 **비평만(수정 X)** 한다.

03 어떻게 부른다

각 리뷰어가 자기 pane에서 `opencode run -m provider/model "메시지"`로 모델을 비대화 호출한다. `opencode.json`이 read-only라 파일을 고치지 못한다.

04 왜 이 조합인가

GPT·North는 **Claude**와 계보가 다르다 → 독립적 시각으로 리뷰한다. North는 무료·코드특화라 빠르다. (모델 식별자는 이 자료 기준 예시.)

opencode run -m 으로 GPT·North 비대화 실행

리뷰어가 자기 pane에서 모델을 부르는 방법이다. -m으로 모델을 고르고 프롬프트를 바로 넘긴다. opencode 설치와 provider 인증이 선행돼야 동작한다.

```
# ask the GPT reviewer (non-interactive, read-only)
timeout 120 opencode run -m openai/gpt-5.4 \
  "Review SPEC.md and solo/solution.py for edge cases"

# ask the North reviewer (free, inline code + NO tools)
timeout 60 opencode run -m opencode/north-mini-code-free \
  "do not read files or use tools; here is the code: ..."

# NOTE: this is anomalyco 'opencode run',
# not the retired Go 'opencode -p'
```

01 전제

opencode가 설치돼 있고 provider 인증이 끝나야 `-m provider/model` 이 동작한다. GPT는 `opencode auth` 로그인, 무료 North는 키가 필요 없다.

02 opencode run

프롬프트를 바로 넘기는 비대화 실행이다. 대화 세션 없이 한 번 묻고 답을 받으며, `timeout` 으로 한 번만 포그라운드로 돌린다.

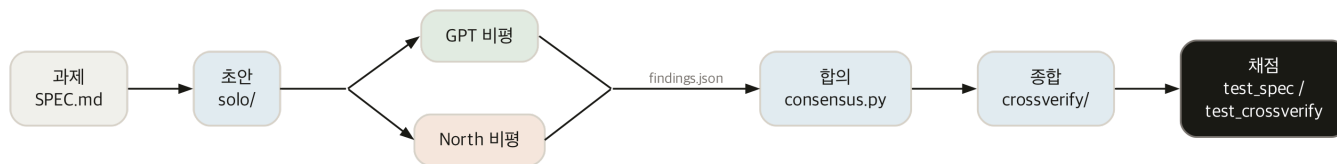
03 North는 코드 인라인 + 도구 금지

`opencode/north-mini-code-free` 는 무료·코드특화 모델이다. 프롬프트에 "파일 읽지 말고 도구 쓰지 마라"를 넣고 코드를 인라인으로 붙이는 것이 핵심이다 — 안 넣으면 레포를 grep·재시도하며 헛돈다. 인라인+no-tools면 약 10초다.

04 read-only · 주의

리뷰어는 `opencode.json` 으로 edit·bash가 deny된 분석 전용이라 파일을 고치지 못한다. anomalyco의 `opencode run` 이며 폐기된 Go판 `opencode -p` 와 다르다.

과제 → 초안 → 독립 비평 → 합의 집계 → 종합 → 두 축 채점



교차검증 루프 — 단계마다 만지는 파일(solo→findings→consensus→crossverify→test)을 표기했다.

01 그림에서 보이는 것

왼쪽 과제(SPEC.md) 에서 출발해 오른쪽 두 축 채점 (test_spec·test_crossverify) 으로 흐르는 한 줄 시퀀스. 각 칸 아래에 그 단계가 만지는 파일을 적었다.

02 어떻게 도나

리드가 초안(solo/solution.py) 을 쓰면 GPT·North가 각자 독립으로 비평하고, 그 비평이 findings.json 으로 모여 consensus.py 로 합의를 집계한다.

03 종합 판정 기준

consensus.py 가 반례 입력 완전일치로 2+ 모델이 겹치면 합의(우선 반영), 1 모델이면 단독(참고)으로 가른다. 서로 다른 버그면 합의 0건이라도 상보적 커버리지가.

04 어디에 쓰나

리드가 합의·단독·자기 판단을 종합해 crossverify/solution.py 를 만들고, 명세 기준선과 교차검증 엡지 두 테스트로 채점한다.

작은 빌드 과제에서 단독이 놓치는 함정

작은 함수 하나로도 단독 에이전트의 사각지대가 드러난다. `parse_duration` 빌드 과제로 확인한다.

과제는 **기간 문자열을 초로 바꾸는** 작은 함수다. 예를 들어 `"30s"` → 30, `"1.5h"` → 5400, `"500ms"` → 0.5, 대문자 `"1H30M"` → 5400, 단위 없는 `"30"` → 오류. 단순해 보이지만 입력 변형이 많아 **구석진 경우**가 곳곳에 숨어 있다. 명세 기준선 테스트로 정상 단위·결합·소수·ms·대문자·공백 처리와 잘못된 입력까지 두루 덮었다.

단독 에이전트의 한 번 풀이는 **명백한 함정은 다 잡았다**. ms와 m을 단위 길이순으로 가린 점, 소수 `"1.5h"`, 대문자 `"1H30M"`, 단위 없는 `"30"`·미지 단위 `"1x"`·음수·빈 값 거부까지 처리했다. 놓친 자리는 **공백 처리**였다 — `text.replace(" ", "")` 로 스페이스를 통째로 지운 탓에, 토큰 사이 탭 (`"1h\t30m"`)이 안 지워져 실패했고 토큰 내부 공백(`"1 h"`·`"1 0m"`·`"500m s"`·`"1h30m s"`)을 오류 없이 잘못 통과시켰다(자체 실측 1건).

채점 테스트(`test_spec.py`)는 초안·비평을 보기 전에 먼저 고정해 점수가 자기충족이 되지 않게 했고, 명세가 침묵하는 회색지대(토큰 내부 공백)는 따로 표시해 한 해석으로 확정했다. 교차검증을 붙이자 계보가 다른 **GPT와 North가 독립적으로** 그 빈자리를 짚었다. 다음 슬라이드에서 두 축 통과 수로, 그다음에 왜 이렇게 되는지로 이어진다.

명세 기준선 vs 교차검증 엣지

결과는 두 축으로 본다 — 먼저 고정된 명세 기준선과, 교차검증이 **추가로** 잡은 엣지. 미묘한 엣지를 한 모델이 빠뜨리기 쉬운 것이 일반 원리이고, **효과 크기는 과제마다 다르다**(자체 실측 1건 예시).

둘 다

통과

명세 기준선 (test_spec)

초안 전에 고정된 명세 기준선은 보통 solo·crossverify가 **둘 다 통과**한다. 단독도 무능하지 않다는 뜻이며, 이 파일은 비평을 본 뒤에도 수정하지 않는다.

추가

엣지

교차검증 (test_crossverify)

리뷰어가 **추가로** 잡은 케이스를 모은 별도 테스트다. 자체 실측 1건에선 단독이 놓친 공백 처리를 두 리뷰어가 메워 **단독 N/M** → **교차검증 M/M**로 차이가 났다.

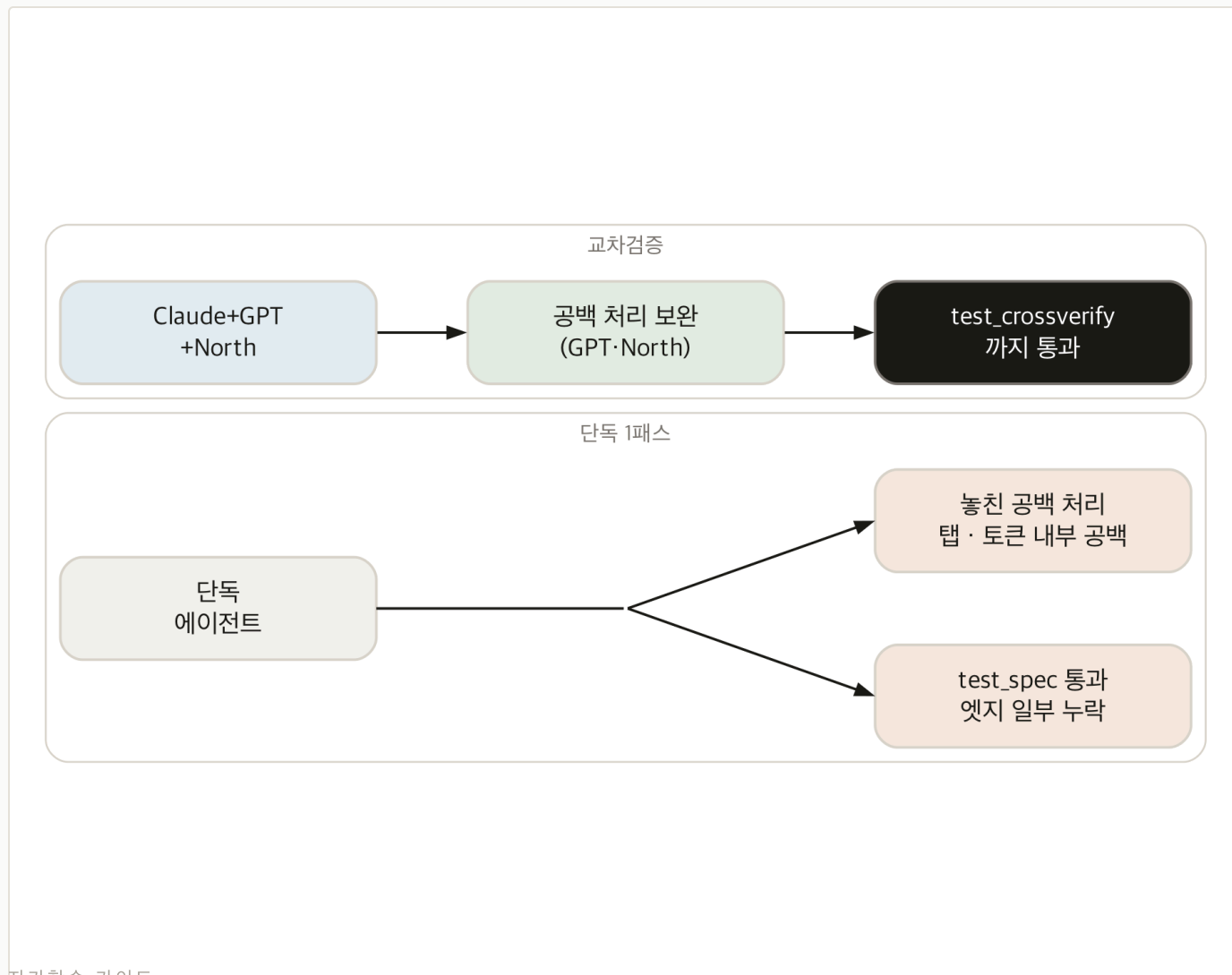
2+

모델 합의

consensus.py

같은 **반례 입력**을 2+ 모델이 지적하면 합의(우선 반영)다. 서로 다른 버그면 합의 0건이라도 **상보적 커버리지**로, 이것도 정당한 성과다.

단독이 놓친 공백 처리 — 어떤 입력에서 터지나



01 그림에서 보이는 것

한쪽 패널은 명세 기준선(**test_spec**) — solo·crossverify가 둘 다 통과, 다른 쪽은 **교차검증 옛지(test_crossverify)** — 리뷰어가 추가로 잡은 케이스다. 단독이 X로 빠뜨린 자리는 모두 **공백 처리**다(자체 실측 1건).

02 토큰 사이 탭에서 터지는 함정

단독은 `text.replace(" ", "")` 로 **스페이스만** 지운다. 그래서 토큰 사이 탭 `"1h\t30m"` 은 탭이 남아 파싱에 실패한다. 모든 공백문자를 소비해야 탭까지 지워진다.

03 토큰 내부 공백에서 터지는 함정

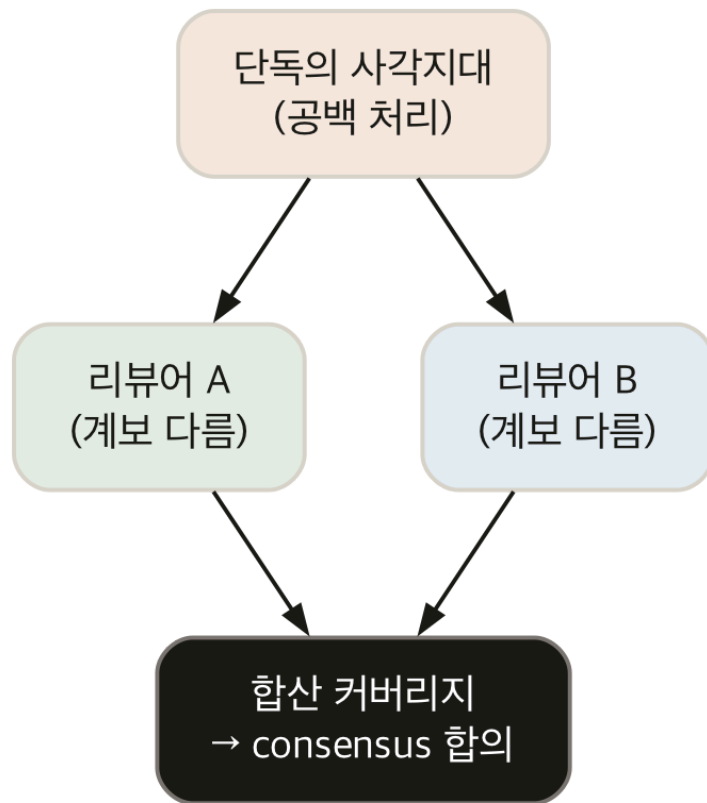
공백을 통째로 지우면 **토큰 내부 공백이 잘못 통과**한다. `"1 h" · "500m s" · "1h30m s"` 는 오류여야 하는데 ValueError가 안 난다. 교차검증본은 토큰 **사이** 공백만 소비해 이를 거부한다.

04 왜 중요한가

혼자선 명백한 함정은 다 잡아도 **더 미묘한 옛지** 한 가지를 놓친다. 이 공백 처리를 GPT·North가 독립적으로 보완해 교차검증 옛지까지 메웠다.

독립적 오답의 비상관성 — 합의는 확신, 불일치는 신호

비상관 오류 — 서로 다른 계보의 모델이 독립으로 틀릴 때, 그 오류가 같은 자리에 겹칠 가능성이 줄어드는 성질.



01 관찰 기반 기대 성질

이건 **이 자료의 관찰에 기반한 기대 성질**이지 외부 벤치마크가 아니다. 채점 테스트는 비평을 보기 전에 먼저 고정해 점수가 자기충족이 되지 않게 했다.

02 독립적으로 빈자리를 메움

이 실습에서 계보가 다른 **GPT와 North가 독립적으로** 같은 미묘한 자리, 즉 **토큰 내부 공백 거부**를 짚었다. 단독이 놓친 공백 처리가 그렇게 메워졌다(자체 실측 1건).

03 합의는 코드로 가린다

합의는 LLM이 아니라 `consensus.py`가 **반례 입력 문자열 완전일치**로 판정한다. 같은 입력을 2+ 모델이 지적하면 합의, 서로 다른 버그면 **상보적 커버리지**다.

04 합의는 확신, 불일치는 신호

여러 모델이 **합의**하면 그 자리는 **확신**, **엇갈리면** 다시 확인할 **신호**다. 명세가 침묵하는 회색지대는 따로 표시해 한 해석으로 확정했다.

챗 대비 토큰 약 4배(에이전트) · 약 15배(멀티에이전트)

Anthropic 데이터(리서치 워크로드 기준)에서 챗 한 번과 견준 토큰 사용량이다. 작업 성격에 따라 달라지며, 더 많이 따질수록 비용도 는다.

4배

에이전트

단일 에이전트가 챗 한 번 대비 쓰는 토큰. 도구 호출과 반복 사고가 더해진 양이다. 리서치 워크로드 기준 수치다.

15배

멀티에이전트

멀티에이전트 시스템이 챗 대비 쓰는 토큰. 워커 수만큼 늘며, 리서치 워크로드 기준이라 작업 성격에 따라 달라진다. 워커를 늘릴수록 토큰이 선형으로 늘어, 단순 작업엔 단일 세션이 경제적이다.

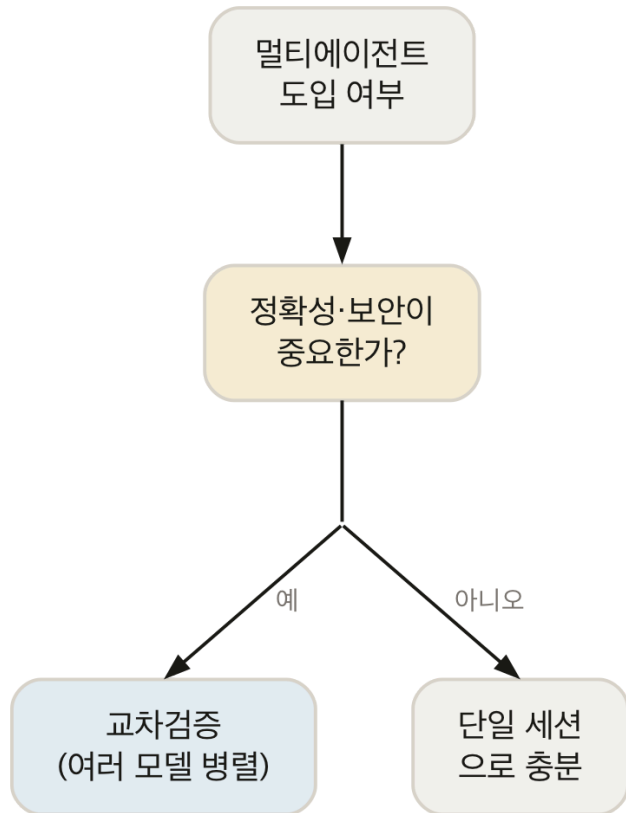
80%

토큰이 좌우

Anthropic 데이터에서, 토큰을 얼마나 쓰느냐가 성능 차이의 상당부분을 좌우한다. 도구 호출 수·모델 선택이 나머지 요인이다.

정확성·보안엔 가치 · 단순 작업엔 과함

비용이 챗의 수 배인 방식이라, 쓸 자리와 피할 자리를 가려야 한다. 갈림은 단 하나 — **정확성·보안이 중요한가**다.



01 무엇을 보나

그림은 한 갈래 질문으로 시작한다. "**정확성·보안이 중요한가?**"에 **예**면 교차검증, **아니오**면 단일 세션으로 흐른다. 이 한 갈림이 도입 여부를 가른다.

02 예 → 교차검증이 가치 있다

미묘한 정확성이 걸린 코드 검토, 보안이 걸린 변경처럼 **한 번의 실수가 비싼** 일이다. 계보가 다른 모델의 **독립 검증**이 사각지대를 줄여, 토큰을 더 쓴 값을 정확성으로 돌려받는다.

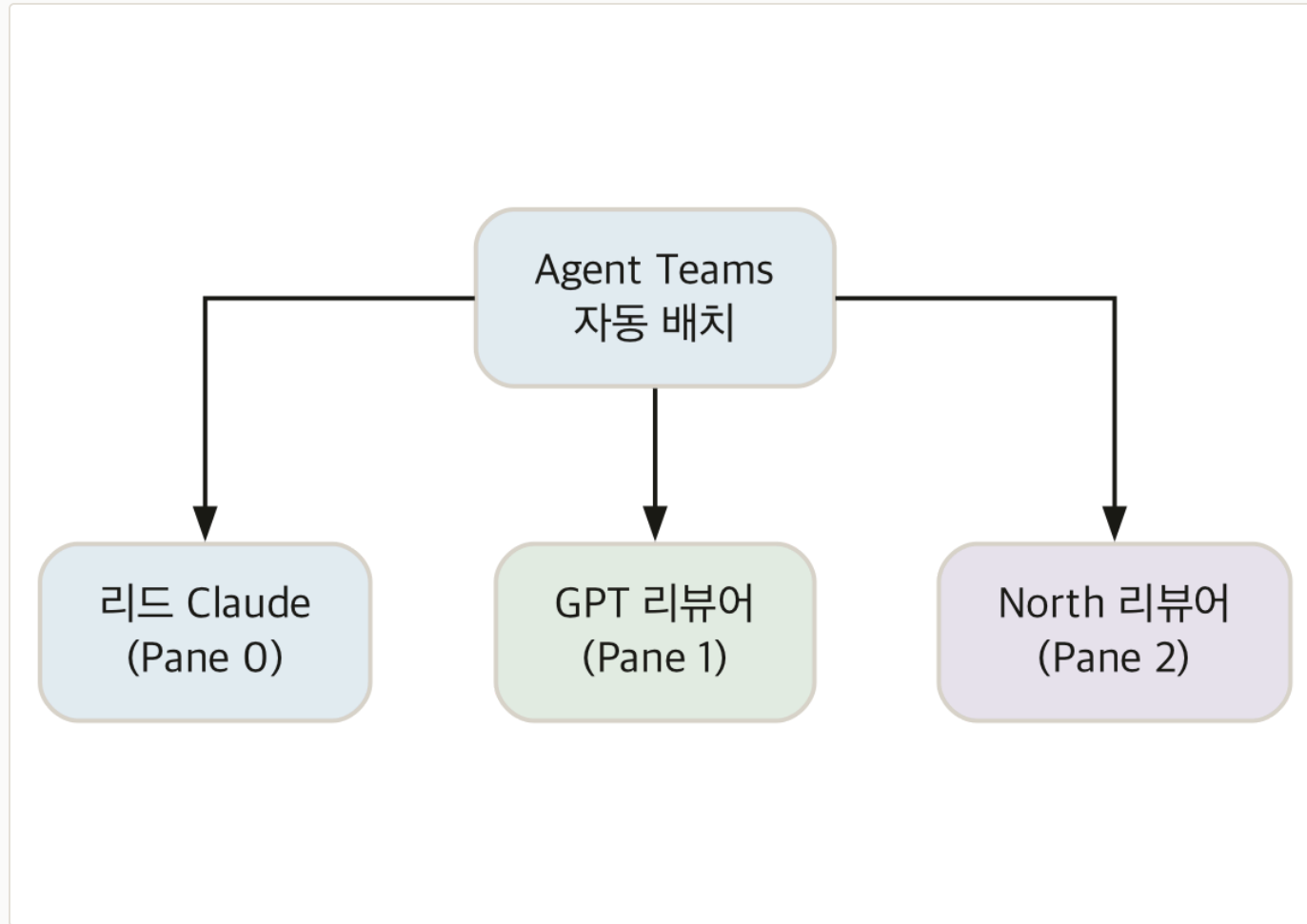
03 아니오 → 단일 세션이면 충분

오타 수정, 짧은 변환처럼 **답이 뻔한** 단순·반복 작업이다. 교차검증의 이득이 거의 없어 단일 세션이 맞다.

04 왜 가려야 하나

멀티에이전트는 챗 대비 **약 15배** 토큰을 쓴다. 단순 작업에 붙이면 **비용만 늘고 이득이 없으니**, 정확성·보안이 걸린 자리에 골라 쓴다.

Agent Teams — 페인 자동 배치 + 공유 task list + 메시지



Claude Agent Teams의 분할 페인 배치와 공유 task list — 이 교차검증 템플릿의 토대.

01 그림에서 보이는 것

이 템플릿이 올라선 토대인 **Claude Agent Teams**다. 팀 리드와 여러 팀메이트가 각자 tmux pane을 차지하고, 리드가 띄운 리뷰어들이 그 위에 붙는다.

02 어떻게 동작하나

분할 페인 모드(`teammateMode: tmux`)가 팀메이트마다 pane을 자동으로 내준다. 리드가 task를 만들어 공유 task list에 올리면, 팀메이트가 각자 일을 **집어(claim)** 처리한다.

03 왜 중요한가

각 task는 **pending** → **in progress** → **completed**로 흐르고, 팀메이트는 `SendMessage` 로 **직접 메시지**를 주고받는다. 실험 기능이라 `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` 로 켜다.

04 경계

task list는 **Claude** 세션끼리 공유된다. 계보가 다른 GPT·North는 opencode를 거쳐 팀메이트로 붙되, 모델 계보가 달라 독립 검증을 더한다.

subagent vs Agent Teams vs 다벤더 교차검증

셋 다 일을 나눈다. 가르는 기준은 워커가 서로 소통하는가, 그리고 계보를 섞는가다. 본류는 Agent Teams 위에 올린 다벤더 교차검증이다.

보고만 한다

subagent (Claude)

구조

한 세션 안에서 돌며 결과를 메인(호출자)에게만 보고한다. 워커끼리는 서로 대화하지 않는다.

맞는 일

검색·탐색처럼 걸가지를 격리해 요약만 받으면 되는 경우. 가볍고 컨텍스트를 아낀다. 모두 같은 Claude 계보다.

서로 소통한다 / ★ 본류

Agent Teams (Claude) · 다벤더 교차검증 (★)

Agent Teams

팀메이트가 공유 task list로 일을 나눠 claim하고 `SendMessage` 로 서로 직접 메시지를 주고받는다. 모두 같은 Claude 세션이다.

★ 다벤더 교차검증(본류)

계보가 다른 GPT·North를 `opencode` 팀메이트로 Agent Teams 위에 올려 같은 과제를 독립 검증한다. 사각지대를 메우는 이 자료의 본류다.

비용

워커 수만큼 토큰이 선형으로 늘어난다. 단순 작업엔 과하다.

팀 규모·운영 제약 (Agent Teams)

이 템플릿이 쓰는 Claude Agent Teams의 팀 규모와 환경 제약이다. 미리 알아두면 막히지 않는다.

팀메이트 수에 정해진 상한은 없지만, 늘릴수록 **토큰 비용이 선형으로** 늘고 누가 무엇을 하는지 맞추는 **조율 비용**도 커진다. 그래서 대부분의 워크플로는 **3~5명으로 시작**하는 편이 균형이 좋다. 이 템플릿은 리드+리뷰어 둘로 단출하게 굴린다. 수보다 **집중**이 중요해, 역할이 또렷한 셋이 흩어진 다섯보다 나은 경우가 많다.

분할 페인 모드는 어디서나 되지는 않는다. 전통적으로 **macOS에서 가장 잘** 동작하고, iTerm2에서는 `tmux -CC`가 권장 진입점이다. 반대로 **VS Code 내장 터미널·Windows Terminal·Ghostty**에서는 분할 페인 모드가 지원되지 않는다.

다만 기본 **in-process 모드**는 어떤 터미널에서도 동작한다. 한 화면 분할이 안 되는 환경이라면 페인 배치를 포기하고 기본 모드로 팀을 굴리면 된다.

핵심 요약과 템플릿으로 시작하기

오늘 잡은 것

- ▶ **Agent Teams + SendMessage** — 리드가 리뷰어를 팀메이트로 스폰하면 각자 pane에 붙고, 비평은 `SendMessage(to:"team-lead")` 로 회신한다.
- ▶ **consensus.py 결정론 합의** — 같은 반례 입력을 2+ 모델이 지적하면 합의, 서로 다른 버그면 상보적 커버리지. LLM 판단은 끼지 않는다.
- ▶ **두 축 채점 · read-only** — 먼저 고정된 명세 기준선과 교차검증이 추가로 잡은 엷지를 나눠 본다. 리뷰어는 opencode가 read-only라 파일을 고치지 못한다.

리드가 Agent Teams로 리뷰어를 팀메이트로 띄워 비평을 받고, `consensus.py` 로 합의를 코드로 집계해 단독이 놓치는 함정을 메운다. 결과는 명세 기준선과 교차검증 엷지, 두 축으로 본다.

직접 해보기

- ▶ `.claude/settings.json` 에 Agent Teams를 켜고, `tmux new -s dev` 로 세션을 만든 뒤 그 안에서 `claude` 를 띄운다.
- ▶ `SPEC.md 구현해줘` 라고 하면 리드가 채점 기준선과 초안을 짜고 GPT·North 리뷰어를 팀메이트로 스폰한다.
- ▶ 비평이 `SendMessage` 로 모이면 `python3 consensus.py findings.json` 으로 합의를 집계하고, 두 테스트로 채점해 차이를 확인한다.